

System Calls for Process Management

Definition

Process-management system calls are system calls used to create, control, replace, wait for, and terminate processes. In POSIX, the main calls are `fork`, `waitpid`, `execve`, and `exit`.

1. Main Process Management Calls

Important Calls

The first major group of POSIX system calls deals with process management:

- `fork()` — create a new child process
- `waitpid()` — wait for a child to terminate
- `execve()` — replace the process image
- `exit()` — terminate a process

2. `fork()` System Call

Meaning of `fork`

`fork()` is the only standard way in POSIX to create a new process. It creates an almost exact duplicate of the calling process.

What `fork()` Copies

After `fork()`, the child gets a copy of:

- file descriptors,
- registers,
- variables,
- process state,
- almost the whole process image.

Parent and Child

After the call, two processes exist:

- the **parent process**,
- the **child process**.

They continue execution separately.

Important Note

At the moment of the fork, parent and child have identical variable values. But afterward, changes in one process do not affect the other, because the data are copied.

Shared Part

The **program text** or code segment is unchanged, so it is shared between parent and child.

3. Return Value of fork()

Return Values

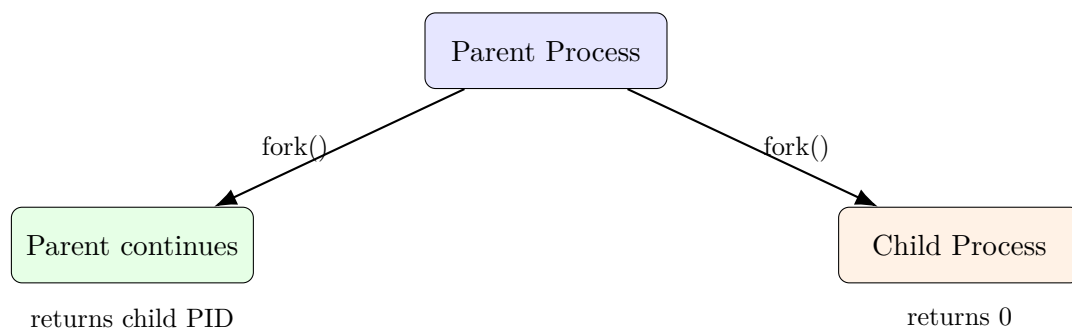
The fork() call returns different values in the two processes:

- in the **child**, it returns 0
- in the **parent**, it returns the child's **PID**

PID

PID means **Process Identifier**. Using the return value, each process can determine whether it is the parent or the child.

3.1 Simple Fork Diagram



4. waitpid() System Call

Meaning of waitpid

waitpid() makes a process wait until one of its child processes finishes execution.

Main Use

Usually the parent uses `waitpid()` after creating a child with `fork()`.

Form

```
pid = waitpid(pid, &statloc, options);
```

Functions of Parameters

- first parameter: which child to wait for
- second parameter: where exit status will be stored
- third parameter: options controlling the behavior

Special Case

If the first parameter is `-1`, then `waitpid()` can wait for **any child process**.

Status Information

When the child terminates, the value stored in `statloc` tells whether termination was:

- normal, or
- abnormal,

and may also contain the exit value.

5. `execve()` System Call

Meaning of `execve`

In most cases, after `fork()`, the child must run a different program. This is done by `execve()`, which replaces the child's current process image with a new program.

Main Idea

`execve()` does not create a new process. Instead, it transforms the existing process into a new program.

Form

```
s = execve(name, argv, environp);
```

Parameters of `execve`

- `name` — name of the program file to execute
- `argv` — pointer to argument array
- `environp` — pointer to environment array

Note

Several library routines such as `execl`, `execv`, `execle`, and `execve` are commonly used. In general discussion, they are often treated together as `exec`.

6. How Shell Uses `fork`, `waitpid`, and `execve`

Shell Behavior

The shell gives a practical example of process-management calls.

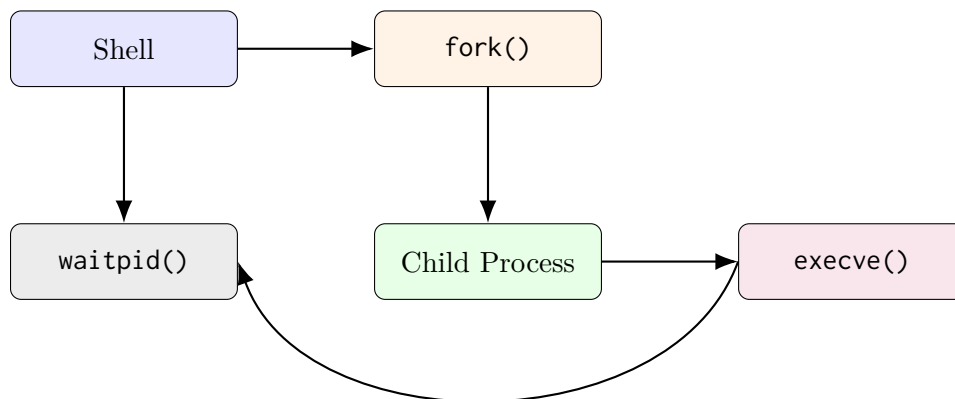
Steps in the Shell

1. shell reads a command,
2. shell calls `fork()`,
3. child process runs the command using `execve()`,
4. parent waits using `waitpid()`,
5. after child finishes, shell reads the next command.

Simplified Shell Code

```
#define TRUE 1
while (TRUE) {
    type_prompt();
    read_command(command, parameters);
    if (fork() != 0) {
        waitpid(-1, &status, 0);
    } else {
        execve(command, parameters, 0);
    }
}
```

6.1 Shell Process Flow Diagram



7. Command-Line Arguments

Example Command

Consider the command:

```
cp file1 file2
```

After `fork()`, the child uses `execve()` to run the program `cp` and passes the source and target file names as arguments.

Main Parameters in C

Most C programs have a main function like:

```
main(argc, argv, envp)
```

where:

- `argc` = number of command-line items
- `argv` = array of command-line strings
- `envp` = environment strings

For `cp file1 file2`

- `argc` = 3
- `argv[0]` = "cp"
- `argv[1]` = "file1"
- `argv[2]` = "file2"

8. Environment Array

Environment

The third parameter, `envp`, points to an array of strings of the form:

`name = value`

These strings pass environment information to programs.

Examples of Environment Information

Environment variables may store:

- terminal type,
- home directory,
- default printer,
- user preferences.

9. `exit()` System Call

Meaning of `exit`

When a process finishes execution, it uses `exit()` to terminate itself.

Form

```
exit(status);
```

Exit Status

The `status` value usually lies between 0 and 255. This value is returned to the parent through `statloc` in `waitpid()`.

10. Process Memory Segments

Memory Layout

In UNIX, a process is divided into three main memory segments:

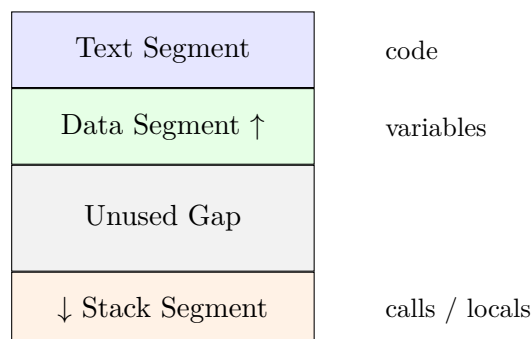
- **text segment** — program code
- **data segment** — variables
- **stack segment** — function calls and local data

Growth Direction

- the **data segment** grows upward
- the **stack segment** grows downward

Between them there is usually an unused gap of address space.

10.1 Simple Segment Diagram



Extra Note

The stack grows automatically as needed. But growth of the data segment is traditionally done explicitly using the `brk` system call. In practice, programmers usually use `malloc()` instead of calling `brk` directly.